true JSON, but each standalone entry is. The following listing shows how the last entry looks once it has been formatted in a more friendly way.

Listing 12.21 The Serilog default JSON formatter output

```
\{
  "Timestamp": "2022-01-08T08:42:50.6276136+00:00",
  "Level": "Information",
  "MessageTemplate": "{count} Cities retrieved",
  "Properties": {
    "count": 12,
    "SourceContext": "CityBreaks.Pages.IndexModel",
    "ActionId": "cda34ae3-2c8f-457a-964b-d7200a0f64df",
    "ActionName": "/Index",
    "RequestId": "0HMEIGBD156VF:00000001",
    "RequestPath": "/",
    "ConnectionId": "0HMEIGBD156VF"
  }
}
```

Note the level of detail here. The property bag, represented by the `Properties` node, includes more information than the console logger. As well as request details and the log category, it includes a property named `count,` which has been taken from your log message template. This enables querying of the property by tools that can parse the JSON log and illustrates why you should use named placeholders in your log message templates instead of interpolation or plain string concatenation.

It is not unusual for users to make mistakes when they modify data within an application. Sadly, I've also found it is not unusual for some users to evade responsibility, blaming someone else or *the system* for their mistake. What would be really useful in these situations would be to show the state of the data before they made an update and then again after the update was made, so there is clear evidence of the changes made and when they were made. In other words, you want a kind of transaction log.